Infranet — Pitch Brief

A marketplace for the compute the world already owns.

Draft for review · 2026-07-13 · a proposal, not a live product or an active fundraise. Canonical path `docs/infranet/INFRANET-COMBINED-BRIEF.md`*; research tree at* `projects/infranet/` *(retained); engineering notes at* `projects/infranet/ARCHITECTURE.md`*; runnable PoC v0 at* `projects/infranet/poc/`*. This revision narrows V1 to the compute marketplace per reviewer feedback; identity and communities move to a clearly-labeled V2 section.*

The problem

The developed world is full of computers that do almost nothing. Gaming PCs that work hard four hours a week. Office desktops that sleep sixteen hours a day. Last generation's laptop in a drawer, the mini-PC bought for one project, the home server that finished its job years ago. Each of these machines is more capable than most of the cloud instances people actually rent, and each sits idle the overwhelming majority of the time.

Meanwhile, the same households and small groups pay rent to run tiny things. A club website. A forty-person game community. A family photo archive. The booking page for a two-chair barbershop. A nightly batch job. None of this needs a datacenter. Almost all of it ends up on one, because hyperscale clouds are priced and designed for enterprises, with billing complexity and surprise invoices to match, and the "free" tier alternatives own the rules, the data, and the exit.

So there is idle supply on one side, priced-out demand on the other, and no venue where they meet. That is not a technology gap; it is a *marketplace* gap. Sellers with inventory, buyers with small budgets, and no eBay between them.

The idea

Infranet is that venue: **a platform for buying and selling shared compute**, the way eBay is a platform for buying and selling goods. Machine owners list spare capacity. Small services and users buy it. The platform does what a marketplace operator does — matching, metering, settlement, and trust — and takes a fee on settled volume. eBay never owned the Beanie Babies; Infranet never owns the machines.

Usage is metered in **compute tokens**, a unit of machine work that behaves the way the watt-hour behaves for electricity: an honest, outcome-neutral measure of work delivered, and the unit in which producers earn and consumers pay.

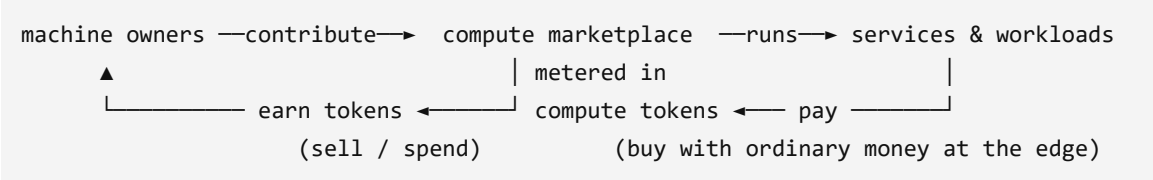
One thing distinguishes this from a build-it-and-they-will-come marketplace, and it is load-bearing: **the company develops a limited set of first-party services on the platform from day one** — small-app hosting, household backup, batch workers — so that early producers have a real buyer before any third party shows up. The marketplace is the product; the first-party services are the seed demand that makes it liquid.

One clarification, because an earlier draft got it wrong: **Infranet is not an LLM platform.** Serving frontier language models takes dense, interconnected datacenter GPUs; attempting it on garage hardware would be emptying a lake with a spoon. An LLM provider could certainly become a *customer* of this marketplace — buying capacity to run tools and host applications, where small local models genuinely fit — but LLM serving is not the point, and the earlier framing of an "LLM gateway wedge" stays withdrawn.

Everything else this platform could carry — identity, age-gated communities, richer service ecosystems — is deliberately out of V1. It lives in the V2 section near the end of this document, after the marketplace it depends on.

How it works, in human terms

Four kinds of participant. **Machine owners (producers)** install an agent that offers spare capacity and earn compute tokens for work actually delivered. **Buyers (consumers)** — small services, households, developers — pay in tokens, whether they earned them by contributing a machine or bought them with ordinary money. **Service builders** (the company first, third parties later) package software to run on the network and price it in tokens. **The company** operates the venue: scheduling, metering, settlement, support. It is funded by a fee on settled volume and by its own first-party services, and it does not buy a hardware fleet; supply is recruited, not purchased.



What a compute token measures

A compute token is a unit of *compute*. It is emphatically not an LLM token, the word fragment language models bill by; the name collision is unfortunate and worth killing early. The right analogy is the watt-hour. An electricity meter does not care whether the kettle boiled water for tea or boiled it dry; it charges for energy delivered. Infranet meters the same way: how much CPU or GPU runtime did this task consume, normalized across hardware by

benchmark tier. A slow core-hour and a fast core-hour are different quantities of work, and the meter knows the difference. Storage and bandwidth are metered beside it in their own native units (GB-month, GB transferred).

Metering runtime does three useful things.

It **rewards computational efficiency**. An application that does the same job in a tenth of the cycles costs its users a tenth as much, so efficient software wins on price. Flat-rate cloud instances quietly reward the opposite: overprovisioning and waste.

It **keeps the platform neutral about outcomes**. The meter charges for work delivered; what the software did with that work is between the application and its users. The hallucination question this raises is answered head-on in the objections below.

It **makes hardware differences priceable instead of fatal**. Because value ties to metered runtime rather than to any particular outcome, machines of very different capability can participate at honest rates, and the producer's real marginal cost, electricity, sets a visible floor. One observation worth logging: where power generation is subsidized or off-peak surplus is real, garage compute gets structurally cheaper, so a government subsidizing power generation is, in passing, subsidizing distributed compute supply. That is an observation about tailwinds, not a plan.

Earning and spending

A machine owner attaches a box; the agent advertises what it can do and when. The scheduler places work on it, meters what runs, and credits tokens. The owner spends those tokens on services their own household uses, or sells them to people who arrived with money instead of hardware. The daily loop should feel like a prepaid meter rather than a trading floor. Cash touches the system only at the edges, through payment rails covered below.

Running strangers' code on volunteers' machines

This is the marketplace's hardest engineering problem, and the pitch is honest about how it is handled. Fuller working notes: `projects/infranet/ARCHITECTURE.md`.

Isolation: a VM airgap, not containers. Every third-party job runs inside a virtual machine on the host — the boundary that lets the platform tell a machine owner "a stranger's workload cannot touch your files or your LAN." The practical direction is micro-VMs (the Firecracker / Cloud Hypervisor class: sub-second boot, megabytes of overhead, built for multi-tenant isolation on thin hardware — the same technology AWS runs Lambda on). Containers were considered and rejected as the primary boundary: lighter, but they share the

host kernel, and one kernel exploit hands the payload the machine. Kubernetes-style orchestration was likewise noted and set aside as too heavy for part-time volunteer nodes; the coordinator must be much lighter.

The threat model: "pay someone to run your virus." The defining abuse case is an attacker who pays, honestly, to have malware executed on thousands of strangers' machines. The defense is layered, assuming each layer fails sometimes. Payloads are **pre-screened before dispatch** on the requester's side of the pipeline — static analysis, signatures, anomaly scoring — which raises attacker cost but costs compute overhead and will produce false negatives. When screening misses, the **VM airgap is the backstop**: the payload detonates inside a guest with no host filesystem, no credentials, and a policed network path. Around both sits accountability: jobs arrive from funded, identified accounts, and abusive requesters lose deposits and access. A **restricted tier** is also on the shelf: a dispatcher that ships only pure math operations — whitelisted numeric kernels, no syscalls, no I/O — collapsing the attack surface to near zero at the price of a much smaller addressable market; useful as the default tier for first-time volunteer machines, not as the whole platform.

Networking. Guest traffic is routed — requester to platform to resource host to VM — with the guest's egress passing back through host-side network control, where policy can throttle, log, or deny. The exact sandbox network policy (default-deny vs opt-in connectivity as a priced capability) is an open design item, flagged rather than hand-waved.

Geography and latency. Cross-region round trips are brutal for interactive work; a scheduler that pairs a consumer in India with a producer in the US has failed that user at any price. The design borrows the availability-zone idea from AWS: supply and demand are grouped into **latency domains** and matched within a domain by default, crossing domains only for batch work that does not care. Predicting latency from observed network history would improve placement, but a fine-grained latency map of a volunteer's connection is also a partial map of where they live — a privacy tradeoff this design flags explicitly and resolves toward coarse zone labels and aggregated statistics rather than per-host telemetry.

Cold start. A marketplace with no producers has nothing to sell, and critical mass of supply is the historical killer of every project in the prior-art table below. The plan attacks it from both ends — first-party services as anchor demand, supply recruited from the homelab and self-hosting communities that already run always-on hardware for fun — and launches one latency domain at a time rather than thinly everywhere, because liquidity has to exist per zone, not in aggregate.

What the company builds first

A marketplace with no listings is a parking lot. The first-party lineup, chosen so each service has a small unit of work, tolerates heterogeneous hardware, and addresses someone who already feels the pain:

- **Small-app hosting.** The club site, the booking page, the game server, the newsletter. Boring, replicated, metered.
- **Household services.** Photo and file backup replicated across a family's own machines plus the network.
- **Automation workers.** Scheduled jobs, scrapers, agents, media transcodes: batch work that tolerates node churn gracefully and soaks up off-peak capacity.

Each of these is a real buyer of marketplace capacity from day one, priced in tokens like any third-party service will be. (The flagship *community* product — spaces with enforceable age gates — depends on the identity layer and is deliberately V2.)

Money: piggyback, do not mint

Infranet should not invent a currency, and does not need to. Compute tokens are an internal metering unit, like prepaid credits. Where real money enters or leaves, the plan is to ride the agentic-commerce wallet rails that emerged in 2025–2026 instead of building payments from scratch:

- **Skyfire** operates payment infrastructure for AI agents around its KYAPay protocol: identity-linked, signed JWT tokens that carry both "who is this agent/party" and "what are they authorized to spend" (skyfire.xyz, KYAPay for A2A). Backers include a16z CSX and Coinbase Ventures (eco.com guide).
- **Google AP2** (Agent Payments Protocol), announced September 2025 with 60+ partners including Mastercard, PayPal, and American Express, handles authorization: three cryptographically signed mandates (intent, cart, payment) prove a human actually approved a transaction; governance has moved to the FIDO Alliance, and the protocol is payment-method agnostic across cards, bank transfers, and stablecoins (comparison, merchant guide).
- **OpenAI/Stripe ACP** (Agentic Commerce Protocol) standardizes checkout inside AI surfaces using single-use, amount-bound Shared Payment Tokens. It shipped with Etsy in September 2025; OpenAI reworked its shopping surface in March 2026, a reminder that this layer is young and still moving (landscape survey).
- **Dollar-pegged stablecoins** are being tested as machine-to-machine settlement right now (Coinbase's x402 pays in stablecoins over HTTP), which is the relevant context for paying thousands of small producers programmatically.

The strategy is to keep compute tokens as metering and settle at the edges through whichever of these rails wins in each geography. One practical asset: through professional work alongside the Skyfire team (via Cequence), the founder's family has a direct, warm introduction path when the settlement conversation starts.

There is also a deeper fit here, and it points forward to V2. These protocols exist because payment networks need to know *which agent* is spending and *on whose authority*: identity attached to money. The identity layer planned for V2 is the same primitive attached to

people. A network that can vouch "this is one verified adult human" composes naturally with payment rails built to ask exactly that kind of question — one more reason identity is the right second act for this platform rather than a distraction from the first.

The objections, head-on

"Pooling spare compute has been tried for twenty-five years and never became a business." Mostly true, and the failure modes are instructive; the prior-art section below goes project by project. The short answer: every prior attempt either gave compute away as charity (which saturates at the limits of altruism) or listed raw commodity FLOPs and waited for buyers who never came, because on datacenter reliability terms the datacenter always wins. Infranet changes the demand side and the workload selection: the company is customer number one for its own marketplace, buying capacity for services people already pay for, and the marketplace sells what garage hardware honestly delivers — small services, batch, replicated storage — rather than pretending to be AWS.

"You're asking volunteers to run strangers' code — malware included." Yes, and that risk is named as the core abuse case rather than buried: pre-screening before dispatch, a VM airgap as backstop when screening fails, funded and identified requester accounts, and a restricted pure-math tier for the most cautious supply. The isolation section above and `ARCHITECTURE.md` carry the detail. No layer is trusted alone.

"If a prompt is bad, or an LLM hallucinates and burns an hour of compute, who eats the cost?" The meter does not adjudicate quality, by design. The power company charges for the kettle you boiled dry. Runtime consumed is runtime paid for; whether the output was worth it is the application layer's responsibility, handled the way software already handles it: budgets and caps, retries, refund policies, reputation. An application that routinely wastes its users' tokens loses its users, and the runtime meter is exactly what makes that waste visible enough to punish. This division of labor is also what keeps accounting stable across wildly different hardware; the moment the meter starts judging outcomes, every dispute becomes a metaphysics seminar.

"Why would anyone attach their PC?" Because the hardware is already bought and the earnings are real, if modest: tokens that pay for the services their own household uses, or convert out through the payment rails. The seed population does this today for free; the self-hosting and homelab communities run exactly these services on exactly this hardware for the satisfaction of it. Offering them compensation and a coordination layer is an easier ask than inventing a behavior from nothing.

"Is home hardware reliable enough?" For the chosen workloads, with replication, yes. Small services, batch work, and storage tolerate individual machines vanishing; the scheduler's job is to make one flaky node a non-event. What garage hardware cannot honestly offer is five-nines, low-latency enterprise SLA, so Infranet does not sell that, and workload selection is a design discipline rather than an apology.

"Is this a crypto scheme?" No public token sale, no yield theater, no speculation machinery. The token is a metering unit. Money enters and exits through regulated wallet rails and stablecoin infrastructure built by others. Whether the internal ledger ever needs a blockchain is an engineering decision for the phase where third parties demand neutrality; a database serves until then.

Prior art, honestly, and why now

The graveyard is real, and anyone pitched this idea should ask about it.

Project	Model	What happened
SETI@home / BOINC (1999–)	Volunteer science compute	Proved millions will donate cycles to a cause; participation saturates at the limits of altruism, and no market formed (Arkhai)
Folding@home	Volunteer science compute	Same lesson; surges on big moments, fades after
Golem (2016–)	Token marketplace for raw compute	Generic compute found thin demand and commoditized
Render Network	GPU marketplace for rendering	Found a real niche and stayed one
Akash, io.net, et al.	Token cloud/GPU marketplaces	Heterogeneous supply loses to datacenters on production reliability; price-sensitive short jobs dominate (field evaluation, structural analysis)

What is different in this design, stated plainly:

1. **Demand is first-party from day one.** The company builds the initial services itself, so baseline capacity has a buyer before any third party shows up. Prior networks launched supply and waited. Cold start is still acknowledged as the single most likely failure mode; this is the mitigation, not a dismissal.
2. **Workload selection matches the hardware.** Small services, batch, and replicated storage are what garage machines honestly deliver. Prior marketplaces listed generic FLOPs and invited comparison with AWS on terms they could only lose.
3. **The isolation technology matured.** Micro-VM hypervisors (Firecracker, Cloud Hypervisor) made strong per-job isolation on modest hardware practical — sub-second boot, tiny overhead. The 2000s volunteer projects ran trusted scientific code; the 2010s marketplaces mostly punted on isolation. Running *untrusted* commercial work on volunteer machines only recently became a sane proposition.
4. **The missing money rails now exist outside the project.** Agent-native wallets and identity-linked payment protocols (Skyfire, AP2, ACP) arrived in 2025–2026, so settlement and payout do not have to be invented in-house, which is where earlier token projects burned years.

None of this guarantees the outcome. It changes which failure modes apply, and the proof-of-concept below is designed to test the new ones cheaply.

Proof of concept: what runs today, and the next step

Honesty first: there is no network today. But as of this revision the core marketplace loop **runs end-to-end on one machine**, at `projects/infranet/poc/`: a submitted job executes in a sandboxed subprocess, a meter reads CPU and wall milliseconds, and a SQLite double-entry ledger debits the consumer and credits the producer in compute tokens — with an assert-based smoke test that fails if the metering or the ledger math breaks. A demo run meters a busy-loop job, a sleeper (billed almost nothing: the meter correctly separates CPU from wall time), and a crashing job (billed for the runtime it burned — the kettle boiled dry), and prints a settlement statement per account.

The PoC's placeholders are documented in its README rather than hidden: isolation is a process boundary standing in for the micro-VM airgap, metering is self-reported from inside the sandbox (production meters from the host side), and there is no multi-machine network yet. The older material in the research tree (in-memory blockchain, identity-registry, and marketplace demos in which every task succeeds unconditionally; FHE/MPC/ZKP as type labels; an unbuilt Rust skeleton — traced in `BASELINE.md`) remains what it was: sketches.

One adjacent thing is real, and it is the reason for conviction: the founder's homelab already runs a public website, admin dashboards, always-on agent processes, and a game-campaign platform on a 2 GB ARM single-board computer plus idle time on a desktop PC. That is daily, lived evidence that useful small services thrive on leftover hardware, along with the operational scar tissue of keeping such services alive.

PoC v1 (one quarter, nights-and-weekends scale, existing hardware, roughly zero cash):

- Three to five volunteer machines across at least two households, running a supervision daemon and a simple scheduler.
- Three real workloads placed and replicated across them: one small hosted app, one household backup target, one batch worker.
- Runtime metering per machine and a monthly settlement statement per owner, on the same SQLite ledger design the v0 already exercises. No blockchain, no token sale, nothing public.
- Exit criteria: every workload survives any single machine going offline; statements reconcile with observed runtime within a few percent; at least one participant chooses to pay money, or contribute capacity, for a second month.

Everything in the PoC is boring on purpose. The claims that are actually novel (runtime metering across heterogeneous boxes, garage supply with real isolation) get tested; the speculative machinery (custom chains, exotic cryptography, identity) stays on the shelf until the platform it would sit on exists.

Business model, briefly

Two revenue streams in V1, in the order they arrive. **First-party service subscriptions**: households and small groups pay monthly for hosting, backup, and automation, in money or in contributed capacity. **A platform fee on settled token volume**: small, and meaningful only at scale, which is why it is listed second — this is the long-term eBay-shaped revenue, and it matures as third-party volume grows. (V2 adds identity verification fees; see below.)

The cost side is software, coordination hosts (small; the current homelab pattern extends a long way), and support. Deliberately absent: a GPU fleet and its refresh cycle. Supply is recruited, machine by machine, and paid only for work delivered.

Sizing is stated honestly: small-service hosting and batch compute are real and growing markets, but a credible bottom-up estimate of the reachable slice is post-PoC work. Early revenue is measured in thousands per month across tens of customers, and the plan treats that as the honest starting point rather than a footnote under a billion-dollar TAM slide.

V2 — identity and walled-garden communities (deliberately deferred)

Everything in this section is roadmap, not V1 scope. It is here because it is where the platform goes once the marketplace works — and because the demand signal is loud enough to plan for.

The regulatory gap. The internet still has no native way to prove that someone is a real, single human being of a certain age without photocopying their life for every website that asks, and regulators have stopped waiting. Australia's under-16 social media ban took effect on 10 December 2025, with penalties up to A$49.5 million (eSafety Commissioner); UK services have run "highly effective" age assurance under the Online Safety Act since July 2025; similar laws are moving through US states and the EU (overview). Today every platform solves this alone and badly: users upload identity documents to dozens of separate vendors, while small communities that want to be adults-only, or teens-safe, have no realistic way to check anyone at all.

The V2 identity layer answers a narrow question well: *is this one real human, and are they over 16, 18, or 21?* Verification happens once, against strong evidence (document check, bank or carrier attestation, in-person options later), and issues a reusable credential that answers yes or no without handing over a name, a birthdate, or a passport scan. The cryptographic ambitions in the research tree (zero-knowledge proofs, multi-party computation, homomorphic encryption — `projects/infranet/PLANNING.md`) all serve exactly this property and remain research; a first version can deliver a useful weaker form with plain signed credentials and upgrade the cryptography as it matures.

Walled-garden communities are what identity makes possible on top of the V1 platform: community spaces that declare a door policy — verified humans only, 16+, 18+, 21+, invite-only, or open — with the platform enforcing the door and the community setting its own rules and owning its own data, running on compute its members may themselves be supplying. This is precisely the capability regulators are now forcing every platform to improvise separately, offered once, as infrastructure. It is also the natural flagship among the "other services" V2 opens up: the compute comes from V1, the door comes from V2, and the service composes both.

Why it belongs on this platform rather than anywhere else: a gated community needs both a place to run and a bouncer, and both have to trust the same fabric — today that is two vendors, two integrations, and two privacy disasters. And as noted in the money section, the wallet rails this platform settles through are themselves identity-attached; the primitives rhyme. V2 adds **identity verification fees** (per-verification or per-seat) as a third revenue stream.

Keeping all of this out of V1 is deliberate: the reviewer feedback that shaped this revision was, in effect, *keep the problem focused — be eBay for shared compute, and do not confuse the pitch with identity*. The marketplace has to work first. This section exists so that when it does, the second act is already designed.

Roadmap and the ask

Phase	Scope	Exit test
V1.0 — PoC (one quarter)	v0 loop (runs today) → 3–5 machines, 3 workloads, metering + statements	PoC v1 criteria above, met and written up
V1.1 — Private beta	20–50 machines from friends, family, homelab communities; first-party services in daily use	≥5 active paying customers across ≥2 services
V1.2 — Settlement + fee	Wallet-rail settlement pilot (open the Skyfire conversation); platform fee switched on	First real-money settlement through an external rail
V1.3 — Third-party services	Registry + SDK so outside builders can price in tokens	≥2 external services settling on the platform
V2 — Identity + communities	Verification-partner integration; age-gated community spaces (16+/18+/21+); identity fees	Compliance-grade age gate live in ≥1 real community

Two standing gates carry over from earlier review: no public token sale or open on-ramp without legal review, and nothing is promoted to production without explicit sign-off (`projects/infranet/AGENT-CHARTER.md`).

The ask, today, is three things. First, agreement on the narrowed scope: V1 is the compute marketplace, full stop; identity and communities are V2. Second, a time-boxed lane to take the running PoC v0 to the multi-machine PoC v1 on existing hardware. Third, two or three design-partner customers for the first-party services, and consent to use the Skyfire introduction when the settlement phase approaches. External capital is a beta-phase conversation, and it should happen with PoC data in hand rather than with this document alone.

Sources

Structure and pitch-memo form:

- Y Combinator Series A guide, investment memo template — via visible.vc and the YC guide text

Isolation and micro-VMs:

- Firecracker micro-VM project (AWS; the Lambda/Fargate isolation layer)
- Cloud Hypervisor project

Wallet protocols and agentic commerce:

- Skyfire and KYAPay A2A extension
- Agentic commerce standards: UCP vs ACP vs AP2 (2026 merchant guide)
- Agentic payments protocols compared (Crossmint)
- What is agentic commerce? The 2026 guide (eco.com)
- Agent payment protocols landscape survey (GitHub)

Prior art on shared/idle compute:

- Tokenizing idle compute (Arkhai)
- Why current distributed compute marketplaces are broken (Arkhai)
- Why GPU marketplaces fail production workloads (dev.to)

Age-verification regulation (V2 section):

- Australia social media age restrictions (eSafety Commissioner)
- Online age verification laws by country (Wikipedia)

Internal sources: `projects/infranet/` R&D tree (ARCHITECTURE, PLANNING, RESEARCH, COMPUTE_REWARDS, COST_ANALYSIS, BLOCKCHAIN_PLATFORM, DEMO, poc/), home-server `BASELINE.md` source trace (2026-07-11), founder/engineering discussion on isolation and threat model (2026-07, paraphrased), and the prior combined brief revisions (in git history).

Document control

Field	Value
Title	Infranet — Pitch Brief (V1 marketplace / V2 identity revision)
Canonical path	`docs/infranet/INFRANET-COMBINED-BRIEF.md`
Supersedes	2026-07-13 story-first rewrite (this same path, prior revision): scope narrowed to V1 = compute marketplace per reviewer feedback; identity/communities moved to V2 section; engineering discussion folded in; PoC v0 now runs
Status	Proposal / brief only; nothing described here is built beyond the PoC v0 loop and the placeholder demos noted above
Feedback	`AI_GROUPCHAT.md` ledger, or project `infranet` in `agents/user-tasks.json`